OS ASSIGNMENT 4

Name-PRADYUMNA P

SRN-PES2UG2CS359

1A)

```
pradyumna_p@LAPTOP-8MLIUB35:~/PES2UG24CS359-pes-vcs$ nano object.c
pradyumna_p@LAPTOP-8MLIUB35:~/PES2UG24CS359-pes-vcs$ make test_objects
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -o test_objects test_objects.o object.o -lcrypto
pradyumna_p@LAPTOP-8MLIUB35:~/PES2UG24CS359-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
pradyumna_p@LAPTOP-8MLIUB35:~/PES2UG24CS359-pes-vcs$
```

1B)

```
pradyumna_p@LAPTOP-8MLIUB35:~/PES2UG24CS359-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
pradyumna_p@LAPTOP-8MLIUB35:~/PES2UG24CS359-pes-vcs$
```

2A)

```
pradyumna_p@LAPTOP-8MLIUB35:~/PES2UG24CS359-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
pradyumna_p@LAPTOP-8MLIUB35:~/PES2UG24CS359-pes-vcs$
```

2B)

```
pradyumna_p@LAPTOP-8MLIUB35:~/PES2UG24CS359-pes-vcs$ xxd .pes/objects/24/38333fa32791bbc3ef
16944c1d73081533bb9f7d8898e7427992f6e59dccf5
00000000: 7472 6565 2031 3435 0031 3030 3634 3420  tree 145.100644
00000010: 6279 652e 7478 7400 b17b 838c 5951 aa88  bye.txt..{..YQ..
00000020: c096 35c5 895e f7e0 8f7f a197 4d90 1ce2  ..5..^......M...
00000030: 82f3 0e08 de0c cd92 3130 3036 3434 2066  ........100644 f
00000040: 696c 6531 2e74 7874 0009 f535 569f 594f  ile1.txt...5V.YO
00000050: 75e4 b8b1 86ea 7988 7f08 6987 f19f bca2  u.....y...i.....
00000060: b489 5376 60be 6ab4 3631 3030 3634 3420  ..Sv`.j.6100644
00000070: 6669 6c65 322e 7478 7400 e00c 50e1 6a2d  file2.txt...P.j-
00000080: f38f 8d6b f809 e181 ad02 48da 6e67 19f3  ...k......H.ng..
00000090: 5f9f 7e65 d6f6 0619 9f7f                 _.~e......
pradyumna_p@LAPTOP-8MLIUB35:~/PES2UG24CS359-pes-vcs$
```

3A)

```
pradyumna_p@LAPTOP-8MLIUB35:~/PES2UG24CS359-pes-vcs$ rm -rf .pes
pradyumna_p@LAPTOP-8MLIUB35:~/PES2UG24CS359-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
pradyumna_p@LAPTOP-8MLIUB35:~/PES2UG24CS359-pes-vcs$ echo "hello" > file1.txt
pradyumna_p@LAPTOP-8MLIUB35:~/PES2UG24CS359-pes-vcs$ echo "world" > file2.txt
pradyumna_p@LAPTOP-8MLIUB35:~/PES2UG24CS359-pes-vcs$ ./pes add file1.txt file2.txt
pradyumna_p@LAPTOP-8MLIUB35:~/PES2UG24CS359-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  commit.c
  untracked:  test_objects.c
  untracked:  tree.c
  untracked:  test_tree.c
  untracked:  README.md
  untracked:  index.c
  untracked:  Makefile
  untracked:  pes.h
  untracked:  test_sequence.sh
  untracked:  commit.h
  untracked:  object.c
  untracked:  .gitignore
  untracked:  tree.h
  untracked:  pes.c
  untracked:  test_tree
  untracked:  index.h
  untracked:  test_objects
  untracked:  pes_debug

pradyumna_p@LAPTOP-8MLIUB35:~/PES2UG24CS359-pes-vcs$ |
```

3B)

```
pradyumna_p@LAPTOP-8MLIUB35:~/PES2UG24CS359-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776684350 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776684356 6 file2.txt
pradyumna_p@LAPTOP-8MLIUB35:~/PES2UG24CS359-pes-vcs$ |
```

4A)

```
pradyumna_p@LAPTOP-8MLIUB35:~/PES2UG24CS359-pes-vcs$ ./pes commit -m "Initial commit"
[main 8c4f4ef] Initial commit
Committed: 8c4f4ef85ca2... Initial commit
pradyumna_p@LAPTOP-8MLIUB35:~/PES2UG24CS359-pes-vcs$ echo "more content" >> file1.txt
pradyumna_p@LAPTOP-8MLIUB35:~/PES2UG24CS359-pes-vcs$ ./pes add file1.txt
pradyumna_p@LAPTOP-8MLIUB35:~/PES2UG24CS359-pes-vcs$ ./pes commit -m "Second commit"
[main 642a1e5] Second commit
Committed: 642a1e5c5055... Second commit
pradyumna_p@LAPTOP-8MLIUB35:~/PES2UG24CS359-pes-vcs$ echo "goodbye" > bye.txt
pradyumna_p@LAPTOP-8MLIUB35:~/PES2UG24CS359-pes-vcs$ ./pes add bye.txt
pradyumna_p@LAPTOP-8MLIUB35:~/PES2UG24CS359-pes-vcs$ ./pes commit -m "Add farewell"
[main bd93323] Add farewell
Committed: bd93323f409a... Add farewell
pradyumna_p@LAPTOP-8MLIUB35:~/PES2UG24CS359-pes-vcs$ ./pes log
commit bd93323f409a19557f95c250ef1f9b716f75ddfce510a5d5e08f6882989f4935
Author: Pradyumna_P PES2UG24CS359
Date:   1776684847

    Add farewell

commit 642a1e5c5055f61b0d3fda48eef6ba7221c81d9499773b0a22df33cf04a33aee
Author: Pradyumna_P PES2UG24CS359
Date:   1776684818

    Second commit

commit 8c4f4ef85ca28b660f25608633c27db1125b85a5c90ba97352b8f047f77638f0
Author: Pradyumna_P PES2UG24CS359
Date:   1776684784

    Initial commit

pradyumna_p@LAPTOP-8MLIUB35:~/PES2UG24CS359-pes-vcs$
```

4B)

```
pradyumna_p@LAPTOP-8MLIUB35:~/PES2UG24CS359-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/09/f535569f594f75e4b8b186ea79887f086987f19fbca2b489537660be6ab436
.pes/objects/24/38333fa32791bbc3ef16944c1d73081533bb9f7d8898e7427992f6e59dccf5
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/64/2a1e5c5055f61b0d3fda48eef6ba7221c81d9499773b0a22df33cf04a33aee
.pes/objects/8c/4f4ef85ca28b660f25608633c27db1125b85a5c90ba97352b8f047f77638f0
.pes/objects/af/876eb9ab5c3c5ccd97d1c76e774f1ab4b3788bb0200559a8ff0db9630d8d19
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/bd/93323f409a19557f95c250ef1f9b716f75ddfce510a5d5e08f6882989f4935
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/f5/902ee73b6aa25436f687067fc4cc0cb481cf416740d3c3c3ed6d9c4de7b6c2
.pes/refs/heads/main
pradyumna_p@LAPTOP-8MLIUB35:~/PES2UG24CS359-pes-vcs$
```

4C)

```
pradyumna_p@LAPTOP-8MLIUB35:~/PES2UG24CS359-pes-vcs$ cat .pes/refs/heads/main
bd93323f409a19557f95c250ef1f9b716f75ddfce510a5d5e08f6882989f4935
pradyumna_p@LAPTOP-8MLIUB35:~/PES2UG24CS359-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
pradyumna_p@LAPTOP-8MLIUB35:~/PES2UG24CS359-pes-vcs$
```

FINAL

```
pradyumna_p@LAPTOP-8MLIUB35:~/PES2UG24CS359-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
[main 4fd4e25] Initial commit
Committed: 4fd4e254a5bb... Initial commit

Log after first commit:
commit 4fd4e254a5bb7ca0dd5570e765da3b6dec4cf43ade05720a2f18e00d2e379c49
Author: Pradyumna_P PES2UG24CS359
Date:   1776685045

    Initial commit


--- Second Commit ---
[main c33fdd5] Update file.txt
Committed: c33fdd5e2e69... Update file.txt

--- Third Commit ---
[main 1a0942d] Add farewell
Committed: 1a0942d4cf79... Add farewell

--- Full History ---
commit 1a0942d4cf79b4ac30b28750aa0fb69659e907f7c6973d8ce2ef99a48a7ea535
Author: Pradyumna_P PES2UG24CS359
Date:   1776685045

    Add farewell

commit c33fdd5e2e69c78c3b4b97177c017e0bdfcae6664b3a7a4f6f61b8bbb60a79d0
Author: Pradyumna_P PES2UG24CS359
Date:   1776685045

    Update file.txt

commit 4fd4e254a5bb7ca0dd5570e765da3b6dec4cf43ade05720a2f18e00d2e379c49
Author: Pradyumna_P PES2UG24CS359
Date:   1776685045

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
1a0942d4cf79b4ac30b28750aa0fb69659e907f7c6973d8ce2ef99a48a7ea535

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/1a/0942d4cf79b4ac30b28750aa0fb69659e907f7c6973d8ce2ef99a48a7ea535
.pes/objects/4f/d4e254a5bb7ca0dd5570e765da3b6dec4cf43ade05720a2f18e00d2e379c49
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/c3/3fdd5e2e69c78c3b4b97177c017e0bdfcae6664b3a7a4f6f61b8bbb60a79d0
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
pradyumna_p@LAPTOP-8MLIUB35:~/PES2UG24CS359-pes-vcs$ |
```

Q5.1: A branch in Git is just a file in .git/refs/heads/ containing a commit hash. Creating a branch is creating a file. Given this, how would you implement pes checkout <branch> — what files need to change in .pes/, and what must happen to the working directory? What makes this operation complex?

To implement pes checkout <branch>, the following must happen:

• Files Modified: The .pes/HEAD file must be updated to contain ref: refs/heads/<branch>.

• Working Directory (WD) Update: The system must read the Tree object associated with the latest commit of the target branch. It then recursively replaces the current files in your folder with the versions stored in the object store.

o Files that exist in the target branch but not in the WD are created.

o Files that exist in the WD but not in the target branch are deleted.

o Files that differ are overwritten

Q5.2: Detecting a "Dirty" Working Directory

To detect uncommitted changes without a full scan, we use the Index:

1. WD vs. Index: Compare the mtime and size of files in the working directory against the metadata in the Index. If they differ, compute the hash of the WD file. If $Hash(WD_File) \neq Hash(Index_Entry)$, the file is "dirty."

2. Index vs. Target: Compare the hash in the Index with the hash in the target branch's Tree.

3. Conflict Rule: If a file is dirty in the WD and its content in the target branch is different from the current branch, the checkout must refuse to proceed to prevent data loss.

Q5.3: The "Detached HEAD" State

In a "Detached HEAD" state, you are pointing directly at a commit hash rather than a branch name.

• Commits: You can still make commits. Your new commit will point to the previous hash as its parent, and HEAD will update to point to the new hash. However, no branch pointer moves forward.

• Recovery: If you switch to another branch, the hash of your "detached" commits will disappear from your view. To recover them, you would need to look at the Reflog (a local log of where HEAD has been) or manually search the .pes/objects directory for "orphaned" commit objects that have no branch pointing to them

Q6.1: The Mark-and-Sweep Algorithm

To clean up unreachable objects, Git uses a Mark-and-Sweep algorithm:

1. Mark Phase: Start at all known "roots" (every branch in refs/heads/, all tags, and HEAD). Recursively follow every hash (Commit $\rightarrow$ Tree $\rightarrow$ Blob).

2. Data Structure: A Hash Set or a Bitset is used to store the IDs of all reachable objects. For massive repositories, a Bloom Filter can be used to quickly check membership.

3. Sweep Phase: Iterate through every file in .pes/objects/. If an object’s hash is not in the "reachable" set, delete it.

4. Estimation: For 100k commits, you would likely visit 10^5 commit objects, plus their associated trees and blobs. Depending on the project size, this could easily involve 10^6 to 10^7 total object visits.

Q6.2: Race Conditions in GC

Running GC concurrently with a commit is dangerous because of a potential Time-of-Check to

Time-of-Use (TOCTOU) race condition:

• The Race:

1. User starts a commit: A new file is hashed and written as a Blob object.

2. GC starts: It looks at the current branches and doesn't see any commit pointing

to that new Blob yet (because the Commit object hasn't been written).

3. GC deletes the Blob as "unreachable."

4. User finishes the commit: The Commit object is written, pointing to a Blob that

no longer exists. The repository is now corrupted.

• The Solution: Git avoids this by using a grace period. By default, git gc will not delete any

object that has an mtime (last modified time) newer than 2 weeks. This ensures that "in-

flight" objects are safe from the sweeper.